

外部排序

数据不能全进内存时，怎样减少块 I/O 轮数

408 · 数据结构 Topic DS-12

母问题：当 I/O 远贵于比较，排序模型怎样改写？

外部排序的生成链是

Block-I/O Cost → Initial Runs → Multiway Merge → Pass Count → I/O Tradeoff.

数据先分成能装入内存的初始归并段并在内存中排序，再用 k 路归并逐轮合并。总成本主要由读写块次数和归并轮数决定，不能直接套内部排序的比较次数。

本册定位与 Owner 边界

- **Owns**：初始归并段、置换选择、多路归并、败者树、缓冲区、I/O 轮数与终止条件。
- **Uses**：Atlas Foundation 的 cost vector；存储系统的 block 概念只作为外部接口，不复制文件系统机制。
- **Stops**：内存内局部排序归 DS11；设备调度、缓存一致性和文件系统完整语义不在本册。

目录

1	初始段与多路归并	2
2	I/O 不变量与终止	2
3	代码与测试证据	2
4	复杂度与概念边界	4
5	做题控制协议	4

1 初始段与多路归并

设内存可容纳 M 条记录，先读入至多 M 条并排序，写成一个 **run**。归并时每个输入 **run** 保留一个当前首元素，取最小者输出并补充同一路的下一条；候选结构可用最小堆或败者树。若有 r 个初始段、每轮 k 路，轮数约为 $\lceil \log_k r \rceil$ ，每轮都要读写全部数据。

为什么败者树值得出现

多路归并每输出一条记录都要更新“哪一路最小”。普通堆和败者树都维护候选极值；败者树显式保存比较路径，便于重复更新同一路。二者都不改变多路归并的正确性不变量。

2 I/O 不变量与终止

每个 **run** 的当前指针只向前移动；输出序列始终是所有当前首元素中的最小值；某一路耗尽后从候选集合移除；所有输入耗尽才结束。缓冲区满/空是 I/O 事件，不是记录顺序事件，必须分开书写。块大小、内存缓冲数量和归并路数改变的是轮数与常数，不改变“每一路有序”的前提。

3 代码与测试证据

完整实现位于 `code/ds12_external_sort.hpp`，测试位于 `code/ds12_external_sort_test.cpp`。可执行核心用内存中的 **run** 模拟外部边界，输出合并结果并计算所需轮数。

Listing 1: 有序 runs 的多路归并与轮数估算

```

10 #include <vector>
11
12 namespace ipara::ds12 {
13
14 inline std::vector<std::vector<int>> make_initial_runs(const std::vector<
    int>& values,
15                                                         std::size_t
    memory_capacity) {
16     if (memory_capacity == 0) throw std::invalid_argument("memory capacity
    must be positive");
17     std::vector<std::vector<int>> runs;
18     for (std::size_t begin = 0; begin < values.size(); begin +=
    memory_capacity) {
19         const std::size_t end = std::min(values.size(), begin +
    memory_capacity);
20         runs.emplace_back(values.begin() + static_cast<std::ptrdiff_t>(begin),
21                             values.begin() + static_cast<std::ptrdiff_t>(end));
22         std::sort(runs.back().begin(), runs.back().end());
23     }
24     return runs;
25 }
26
27 inline std::vector<int> merge_sorted_runs(const std::vector<std::vector<
    int>>& runs) {
28     struct Candidate { int value; std::size_t run; std::size_t index; };

```

```

29  auto greater = [] (const Candidate& left, const Candidate& right) {
30      return left.value > right.value;
31  };
32  std::priority_queue<Candidate, std::vector<Candidate>, decltype(greater) > pending(greater);
33  std::size_t total = 0;
34  for (std::size_t run = 0; run < runs.size(); ++run) {
35      total += runs[run].size();
36      if (!runs[run].empty()) pending.push(Candidate{runs[run].front(), run, 0});
37  }
38  std::vector<int> output;
39  output.reserve(total);
40  while (!pending.empty()) {
41      const Candidate current = pending.top();
42      pending.pop();
43      output.push_back(current.value);
44      const std::size_t next = current.index + 1;
45      if (next < runs[current.run].size()) {
46          pending.push(Candidate{runs[current.run][next], current.run, next});
47      }
48  }
49  return output;
50 }
51
52 inline std::size_t merge_passes(std::size_t run_count, std::size_t fan_in) {
53     if (fan_in < 2) throw std::invalid_argument("fan-in must be at least two");
54     std::size_t passes = 0;
55     while (run_count > 1) {
56         run_count = (run_count + fan_in - 1) / fan_in;
57         ++passes;
58     }
59     return passes;
60 }
61
62 } // namespace ipara::ds12
63
64 #endif

```

Listing 2: 空 run、单 run 与多路耗尽测试

```

8  using ipara::ds12::merge_passes;
9  using ipara::ds12::merge_sorted_runs;
10
11 int main() {

```

```
12  const std::vector<int> input{9, 1, 7, 3, 5, 2, 8, 4, 6};
13  const auto runs = make_initial_runs(input, 3);
14  assert((runs == std::vector<std::vector<int>>{{1, 7, 9}, {2, 3, 5}, {4,
    6, 8}}));
15  assert((merge_sorted_runs(runs) == std::vector<int>{1, 2, 3, 4, 5, 6,
    7, 8, 9}));
16  assert((merge_sorted_runs({{}}, {1, 4}, {}, {2, 3})) == std::vector<int>
    >{1, 2, 3, 4}));
17  assert(merge_passes(1, 2) == 0);
18  assert(merge_passes(10, 3) == 3);
19  bool capacity_error = false;
20  try { (void)make_initial_runs(input, 0); } catch (const std::
    invalid_argument&) { capacity_error = true; }
21  assert(capacity_error);
22  bool fan_in_error = false;
23  try { (void)merge_passes(2, 1); } catch (const std::invalid_argument&) {
    fan_in_error = true; }
24  assert(fan_in_error);
25 }
```

```
clang++ -std=c++17 -Wall -Wextra -Werror -pedantic code/
    ds12_external_sort_test.cpp -o /tmp/ds12_test
/tmp/ds12_test
```

4 复杂度与概念边界

阶段	比较/更新	I/O 视角	不变量	边界
初始段	每段内 存排序	读写 N	每段有序	$N \leq M$
k 路归并	每条更 新候选	每轮读写 N	各路首元素候选	空 run
总过程	依赖实现	$O(N \log_k r)$ 轮次量级	指针单调前进	输入耗尽

5 做题控制协议

先标 N, M, k, r ，再算初始段数量和归并轮数；每一轮分别写“读、比较、输出、补位、写”。看到全部数据能进内存，停止套外部 I/O 公式并转 DS11；看到块大小或缓冲区变化，重新计算轮数而非只比较关键字次数。

压缩复原

五问：内存一次容纳多少？初始 run 怎样形成？当前候选来自几路？哪一路耗尽如何退出？总成本按比较还是块 I/O 计？